EPSI LILLE

Master 1 — DévOps

Pipeline as Code

Atelier 2

Du code source au déploiement automatisé

AGENDA DE L'ATELIER

1	Création et Configuration du Projet Next.js
2	Développement et Cycle Git (Versioning)
3	Configuration du Projet sur VS Code
4	Déploiement du projet sur Heroku
5	Configuration de l'Infrastructure Docker & Jenkins
6	Identifiants et Automatisation Pipeline
7	Bilan

1. Création et Configuration du Projet Next.js

Nous allons initialiser un projet Next.js nommé par exemple nextjs-heroku. Cette étape constitue le point de départ de tout l'atelier.

1.1 Initialisation du projet

Ouvrez votre terminal dans votre dossier de travail et lancez la commande suivante :

```
npx create-next-app@latest nom-de-votre-projet
```

> **Important :** Acceptez les options par défaut durant l'installation en appuyant sur Entrée à chaque étape.

1.2 Exécution locale

Une fois le projet créé, entrez dans son dossier et démarrez le serveur de développement :

```
cd nom-de-votre-projet
npm run dev
```

Ouvrez votre navigateur et accédez à l'adresse suivante :

```
http://localhost:3000
```

Vous devriez voir la page d'accueil de votre application Next.js.

1.3 Arrêt du serveur

Pour stopper le serveur de développement, appuyez sur :

```
Ctrl + C    →    puis tapez O (ou Y) pour confirmer
```

2. Développement et Cycle Git (Versioning)

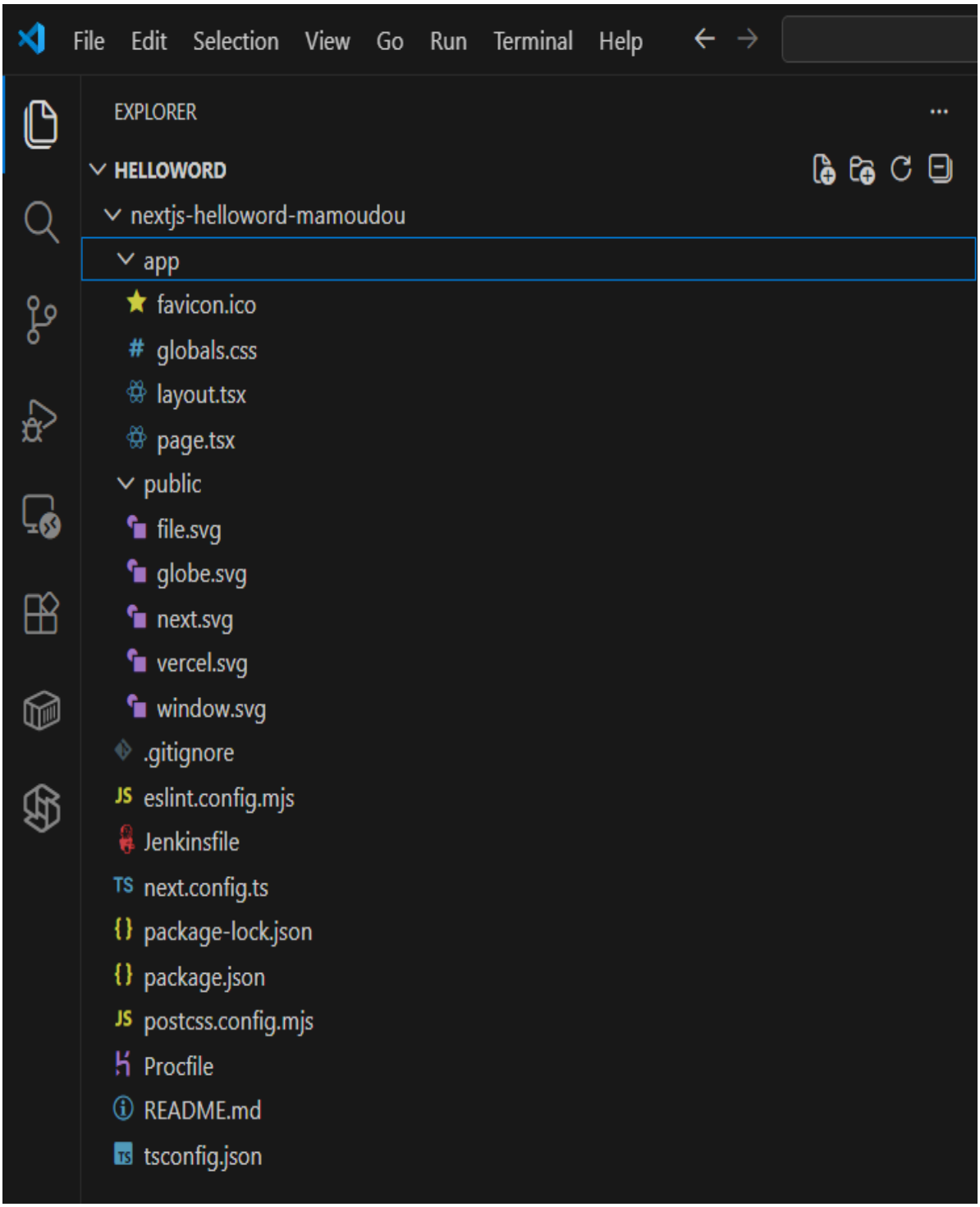
Cette section couvre la gestion du cycle de vie local de l'application Next.js avec Git.

Commande	Description
`npm run dev`	Lance le serveur de développement local sur http://localhost:3000
`git init / git status`	Initialise un nouveau dépôt Git local pour le suivi du projet
`git add .`	Prépare l'ensemble des fichiers modifiés pour l'enregistrement (staging)
`git commit -m "msg"`	Enregistre officiellement les modifications avec un message explicatif
`git branch`	Affiche la branche actuelle du projet (ex. : master ou main)
`git push origin main`	Envoie de modifications

> **Conseil :** Rédigez toujours des messages de commit clairs et en anglais. Ex. : "feat: add login page" ou "fix: resolve navbar bug".

3. Configuration du Projet sur VS Code

Comprendre la structure des fichiers de configuration est essentiel pour maîtriser votre environnement de travail.



3.1 Le cœur du projet

Fichier	Rôle
`package.json`	Carte d'identité du projet : liste les dépendances, scripts npm (dev, build, start) et métadonnées.
`package-lock.json`	Compte-rendu détaillé des versions exactes installées — garantit la reproductibilité entre environnements.
`tsconfig.json`	Manuel de grammaire TypeScript : définit comment transpiler votre code TS en JavaScript lisible par les navigateurs.
`README.md`	Mode d'emploi humain : présente le projet et explique comment l'installer et l'utiliser.
`next-env.d.ts`	Aide TypeScript auto-gérée par Next.js — ne pas modifier.
`layout.tsx ( Racine , squellette de l'appli )`	Ce fichier est le **point d'entrée principal** de votre interface utilisateur. C'est un composant React qui enveloppe toutes les routes de votre application.
`page.tsx ( Page Principale )`	C'est le fichier qui correspond à la route racine / de votre site. Dans Next.js App Router, c'est par défaut un Server Component. Cela signifie que son code est exécuté uniquement sur le serveur (ou au moment du build), et seul le HTML généré est envoyé au navigateur. Cela réduit considérablement le JavaScript envoyé au client.

3.2 Pipeline & déploiement

Fichier	Rôle
`Jenkinsfile`	Robot d'usine : contient les instructions du pipeline CI/CD (test → build → deploy). C'est le fichier le plus important de cet atelier (exploration de cette partie atelier 3)
`Procfile`	Notice de démarrage Heroku : indique à la plateforme la commande à exécuter (ex. : web: npm run start).
`.gitignore`	Liste noire Git : empêche l'envoi de fichiers inutiles ou sensibles sur GitHub (node_modules, .env, etc.).

3.3 Style & affichage

Fichier	Rôle
`globals.css ( styles globaux`	Ce fichier gere le style de toute l'application .Il contient les règles de style générales. Dans un projet Next.js récent, c'est généralement ici que l'on importe les directives de Tailwind CSS (@tailwind base; @tailwind components; @tailwind utilities;) qui vous permettent d'utiliser les classes utilitaires directement dans vos fichiers